

GLOSSAIRE TECHNIQUE

Projet E-Store — Définitions simples et courtes

Module : Développement Full-Stack | Encadrant : Pr. Omar ZAHOUR

Filière : DEV-INFO S6 | Faculté des Sciences Ben M'Sick — Université Hassan II

Année universitaire : 2025–2026

Ce glossaire rassemble les définitions courtes et simples de tous les termes techniques utilisés dans le projet E-Store. Il est organisé par domaines thématiques pour faciliter la lecture et la révision avant la soutenance. Chaque définition est volontairement concise (2 à 3 phrases) afin de permettre une compréhension rapide du concept.

8 Categories	80 Definitions	8 Domaines	2026 Année
-----------------	-------------------	---------------	---------------

A — Backend & Framework Java

Spring Boot	Framework Java qui permet de créer des applications web prêtes à l'emploi rapidement, sans configuration complexe. Il intègre un serveur Tomcat et gère automatiquement les dépendances.	<i>Backend / Java</i>
Spring Security	Module Spring qui sécurise l'application : il gère l'authentification (qui es-tu ?) et l'autorisation (qu'as-tu le droit de faire ?). Il protège chaque endpoint HTTP.	<i>Backend / Sécurité</i>
Spring Data JPA	Module Spring qui simplifie l'accès à la base de données relationnelle. Il génère automatiquement les requêtes SQL à partir d'interfaces Java (Repository).	<i>Backend / Base de données</i>
Spring Data MongoDB	Module Spring qui permet d'interagir avec MongoDB (base NoSQL) de la même manière que Spring Data JPA, sans écrire de requêtes manuelles.	<i>Backend / NoSQL</i>
Maven	Outil de gestion de projet Java. Il télécharge automatiquement les dépendances (bibliothèques) déclarées dans le fichier pom.xml et compile le code.	<i>Backend / Outils</i>
pom.xml	Fichier de configuration Maven d'un projet Java. Il liste toutes les dépendances (libraries), les plugins et les informations du projet.	<i>Backend / Outils</i>
Lombok	Bibliothèque Java qui génère automatiquement le code répétitif : constructeurs, getters, setters, toString, equals via des annotations (@Data, @Builder...).	<i>Backend / Java</i>
@Transactional	Annotation Java qui enveloppe une méthode dans une transaction base de données : si une erreur survient, toutes les modifications sont annulées (rollback).	<i>Backend / Base de données</i>
GlobalExceptionHandler	Classe Spring qui intercepte toutes les exceptions de l'application et renvoie des messages d'erreur JSON uniformes au client, au lieu d'une page d'erreur blanche.	<i>Backend / Architecture</i>
DataSeeder	Composant Spring qui s'exécute au démarrage de l'application pour insérer des données initiales (produits, catégories) dans la base de données.	<i>Backend / Architecture</i>

B — Architecture & Patterns

MVC (Model-View-Controller)	Patron d'architecture qui separe une application en trois parties : Model (donnees), View (affichage) et Controller (logique). Facilite la maintenance et les tests.	<i>Architecture</i>
DTO (Data Transfer Object)	Objet simple utilise pour transferer des donnees entre couches de l'application. Il evite d'exposer directement les entites de la base de donnees a l'API.	<i>Architecture / Pattern</i>
Repository Pattern	Pattern qui isole la logique d'accès aux donnees dans une classe dediee. Le reste de l'application n'a pas besoin de savoir si les donnees viennent d'une base SQL, NoSQL ou d'un fichier.	<i>Architecture / Pattern</i>
Service Layer	Couche intermediaire entre le Controller et le Repository. Elle contient la logique metier (calculs, validations, regles). Chaque service gere un domaine precis (produits, commandes...).	<i>Architecture</i>
Controller	Classe qui recoit les requetes HTTP entrantes, appelle le service adequat et retourne la reponse au client (en JSON). Dans Spring, annotee @RestController.	<i>Architecture / Backend</i>
Entity	Classe Java qui represente une table en base de donnees relationnelle. Chaque instance de la classe correspond a une ligne dans la table. Annotee @Entity en JPA.	<i>Backend / Base de donnees</i>
Domain-Driven Design (DDD)	Approche de conception ou le code est organise autour des domaines metier (customer, catalog, billing...) plutot que des couches techniques. Chaque domaine est autonome et coherent.	<i>Architecture</i>
ACID	Proprietes garantissant la fiabilite des transactions base de donnees : Atomicite (tout ou rien), Coherence, Isolation (transactions independantes), Durabilite (donnees persistantes apres commit).	<i>Base de donnees</i>
Endpoint	URL specifique d'une API qui repond a une action precise. Exemple : GET /api/products retourne la liste des produits. Chaque endpoint correspond a une methode dans un Controller.	<i>API / REST</i>
REST (Representational State Transfer)	Style d'architecture pour les APIs web. Les ressources sont identifiees par des URLs, et les actions sont definies par les methodes HTTP (GET, POST, PUT, DELETE).	<i>API</i>

C — Securite & Authentification

JWT (JSON Web Token)	Token numerique signe utilise pour authentifier un utilisateur. Il contient des informations (utilisateur, roles, expiration) encodees en Base64. Le serveur le verifie sans stocker de session.	<i>Securite</i>
BCrypt	Algorithme de hachage utilise pour stocker les mots de passe de facon securisee. Il ajoute un 'sel' aleatoire et est intentionnellement lent pour resister aux attaques par force brute.	<i>Securite</i>
CORS (Cross-Origin Resource Sharing)	Mecanisme de securite du navigateur qui bloque les requetes HTTP vers un domaine different. Le serveur doit explicitement autoriser les origines frontends (ex: http://localhost:4200).	<i>Securite / Web</i>
Hachage (Hash)	Transformation irreversible d'une donnee (mot de passe) en une chaine fixe. Contrairement au chiffrement, on ne peut pas retrouver l'original. On verifie en hachant l'entree et comparant au hash stocke.	<i>Securite</i>
@PreAuthorize	Annotation Spring Security qui protege une methode : seuls les utilisateurs ayant le role adequat (ROLE_ADMIN, ROLE_USER) peuvent l'appeler. Leve une exception 403 sinon.	<i>Securite / Backend</i>
Filter (Filtre HTTP)	Composant qui s'intercale dans la chaine de traitement des requetes HTTP avant qu'elles atteignent le Controller. Le JwtAuthenticationFilter valide le token JWT a chaque requete.	<i>Securite / Backend</i>
Role	Permission attribuee a un utilisateur qui determine ses droits dans l'application. Dans E-Store : ROLE_USER (client) et ROLE_ADMIN (administrateur).	<i>Securite</i>
Stateless (sans etat)	Architecture ou le serveur ne garde aucune session utilisateur en memoire. Chaque requete doit s'auto-authentifier via son token JWT. Facilite la scalabilite horizontale.	<i>Architecture / Securite</i>

D — Base de Donnees

JPA (Java Persistence API)	Specification Java standard pour mapper des objets Java (entites) vers des tables SQL. Spring Data JPA est l'implementation la plus utilisee.	<i>Base de donnees / Java</i>
ORM (Object-Relational Mapping)	Technique qui fait la correspondance automatique entre les objets Java et les tables SQL. Evite d'ecrire du SQL manuellement pour les operations CRUD.	<i>Base de donnees</i>

MySQL	Systeme de gestion de base de donnees relationnelle (SQL) open-source. Stocke des donnees structurees en tables avec des relations (cles etrangeres). Utilise en production dans E-Store.	<i>Base de donnees</i>
H2	Base de donnees relationnelle en memoire (in-memory), ecrite en Java. Utilisee en mode developpement : aucune installation requise, les donnees disparaissent a l'arret de l'application.	<i>Base de donnees / Dev</i>
MongoDB	Base de donnees NoSQL qui stocke les donnees sous forme de documents JSON (appeles BSON). Adaptee aux donnees flexibles sans schema fixe, comme les avis clients dans E-Store.	<i>Base de donnees / NoSQL</i>
NoSQL	Famille de bases de donnees qui ne suivent pas le modele relationnel SQL. Elles sont adaptees aux grandes quantites de donnees variables et aux structures non tabulaires.	<i>Base de donnees</i>
JPQL (Java Persistence Query Language)	Langage de requetes similaire a SQL, mais qui opere sur les entites Java plutot que sur les tables SQL directement. Utilise avec l'annotation @Query.	<i>Base de donnees / JPA</i>
Migration / Seeder	Script qui initialise ou met a jour la structure et les donnees d'une base. Un seeder insere des donnees de test ou initiales au demarrage de l'application.	<i>Base de donnees</i>
Transaction	Ensemble d'operations base de donnees traitees comme une unite atomique. Si une operation echoue, toutes les precedentes sont annulees (rollback).	<i>Base de donnees</i>

E — Frontend & Angular

Angular	Framework TypeScript developpe par Google pour creer des applications web SPA (Single Page Application). Il structure le code en composants, services et modules.	<i>Frontend / Framework</i>
TypeScript	Langage de programmation base sur JavaScript qui ajoute le typage statique. Il detecte les erreurs a la compilation, avant l'execution dans le navigateur.	<i>Frontend / Langage</i>
Composant (Component)	Unite de base d'Angular. Chaque composant gere une partie de l'interface : un bouton, un formulaire, une page entiere. Il a son propre HTML, CSS et logique.	<i>Frontend / Angular</i>
SPA (Single Page Application)	Application web qui charge une seule page HTML et met a jour dynamiquement le contenu sans rechargement complet du navigateur. Angular est un framework SPA.	<i>Frontend</i>

Standalone Component	Composant Angular 17 qui n'a pas besoin d'être déclaré dans un NgModule. Il importe directement ses dépendances, simplifiant l'architecture de l'application.	<i>Frontend / Angular 17</i>
Signal	Nouveau mécanisme de réactivité dans Angular 17. Un signal est une valeur réactive : quand elle change, les composants qui la lisent se mettent à jour automatiquement, sans RxJS.	<i>Frontend / Angular 17</i>
Lazy Loading	Technique qui charge le code d'une fonctionnalité seulement quand l'utilisateur navigue vers elle. Réduit le temps de chargement initial de l'application.	<i>Frontend / Performance</i>
Route (Routage)	Mécanisme Angular qui affiche un composant différent selon l'URL du navigateur. Exemple : /products affiche la liste, /cart affiche le panier.	<i>Frontend / Angular</i>
AuthGuard	Garde Angular qui protège certaines routes. Il vérifie si l'utilisateur est connecté avant d'afficher une page. Si non, il redirige vers /login.	<i>Frontend / Sécurité</i>
Interceptor	Service Angular qui intercepte toutes les requêtes HTTP sortantes ou les réponses entrantes. Utilisé pour ajouter automatiquement le token JWT à chaque requête (authInterceptor).	<i>Frontend / Angular</i>
Bootstrap 5	Bibliothèque CSS qui fournit des composants graphiques prédéfinis (boutons, cartes, grilles, formulaires). Rend l'interface responsive sur mobile et desktop.	<i>Frontend / CSS</i>
Responsive Design	Conception d'interface qui s'adapte automatiquement à la taille de l'écran (mobile, tablette, desktop). Utilise les grilles CSS flexibles et les media queries.	<i>Frontend / Design</i>

F — Concepts Web & HTTP

HTTP (HyperText Transfer Protocol)	Protocole de communication entre navigateur et serveur web. Chaque échange est une requête (client -> serveur) et une réponse (serveur -> client) indépendantes.	<i>Web</i>
GET	Méthode HTTP pour lire des données. Ne modifie rien sur le serveur. Exemple : GET /api/products récupère la liste des produits.	<i>HTTP / REST</i>
POST	Méthode HTTP pour créer une nouvelle ressource. Les données sont envoyées dans le corps de la requête (body). Exemple : POST /api/auth/register crée un compte.	<i>HTTP / REST</i>

PUT	Methode HTTP pour remplacer completement une ressource existante. Envoie toutes les donnees de la ressource mise a jour.	<i>HTTP / REST</i>
DELETE	Methode HTTP pour supprimer une ressource. Exemple : DELETE /api/cart/items/5 supprime l'article 5 du panier.	<i>HTTP / REST</i>
JSON (JavaScript Object Notation)	Format de donnees texte leger utilise pour echanger des informations entre frontend et backend. Lisible par les humains et les machines.	<i>Web / Format</i>
API (Application Programming Interface)	Interface qui permet a deux logiciels de communiquer. Dans E-Store, le frontend Angular communique avec le backend Spring Boot via une API REST en JSON.	<i>Web / Architecture</i>
Status HTTP	Code numerique qu'un serveur renvoie pour indiquer le resultat d'une requete. 200 = succes, 201 = cree, 400 = mauvaise requete, 401 = non authentifie, 403 = interdit, 404 = non trouve, 500 = erreur serveur.	<i>HTTP</i>
Header HTTP	Metadonnee envoyee avec une requete ou reponse HTTP. Le token JWT est envoye dans le header 'Authorization: Bearer '.	<i>HTTP / Securite</i>
Payload	Donnees utiles transportees dans le corps d'une requete ou reponse HTTP. Aussi le contenu encode d'un JWT (userId, roles, expiration).	<i>HTTP / JWT</i>
Observable (RxJS)	Flux de donnees asynchrone utilise dans Angular. Permet de reagir aux evenements (reponses HTTP, clics) de facon reactive et de les transformer avec des operateurs (map, filter, switchMap...).	<i>Frontend / RxJS</i>
BehaviorSubject	Type special d'Observable RxJS qui memorise la derniere valeur emise et la diffuse immediatement aux nouveaux abones. Utilise pour partager l'etat de l'utilisateur connecte dans AuthService.	<i>Frontend / RxJS</i>

G — DevOps & Outils

Docker	Outil qui permet d'empaqueter une application et ses dependances dans un 'conteneur' portable. Le conteneur s'execute de la meme facon sur n'importe quelle machine.	<i>DevOps</i>
Docker Compose	Outil Docker qui permet de definir et lancer plusieurs conteneurs simultanement via un fichier docker-compose.yml. Dans E-Store : MySQL, MongoDB, PHPMyAdmin, Mongo-Express.	<i>DevOps</i>

Conteneur	Environnement isole et leger qui contient une application et tout ce dont elle a besoin pour fonctionner. Similaire a une machine virtuelle mais beaucoup plus leger.	<i>DevOps</i>
Git	Systeme de controle de version distribue. Il enregistre l'historique de toutes les modifications du code, permet de revenir en arriere et de travailler a plusieurs en parallele.	<i>DevOps / Versioning</i>
GitHub	Plateforme en ligne qui heberge des repositories Git. Permet la collaboration, le partage de code et l'integration continue. Le projet E-Store y est depose.	<i>DevOps / Collaboration</i>
Commit	Instantane du code a un moment donne, enregistre dans l'historique Git. Chaque commit a un message qui decrit le changement effectue.	<i>Git</i>
Branch (Branche)	Copie independante du code sur laquelle on peut travailler sans affecter la branche principale (main). Permet de developper des fonctionnalites en parallele.	<i>Git</i>
npm (Node Package Manager)	Gestionnaire de paquets pour JavaScript/Node.js. Permet d'installer les dependances Angular et les outils frontend declares dans package.json.	<i>Frontend / Outils</i>
Profil Spring (application-dev.properties)	Configuration specifique a un environnement (dev, prod). En dev : H2 en memoire. En prod : MySQL. Active via la variable spring.profiles.active.	<i>Backend / Configuration</i>
JUnit 5	Framework de tests unitaires pour Java. Permet de verifier que chaque methode fonctionne correctement de facon isolee, sans lancer toute l'application.	<i>Tests</i>
Mockito	Bibliotheque Java qui cree des 'faux' objets (mocks) pour les tests unitaires. Permet de tester un service sans avoir besoin d'une vraie base de donnees.	<i>Tests</i>
Test Unitaire	Test qui verifie le bon fonctionnement d'une seule unite de code (methode ou classe) de facon isolee. Dans E-Store : 9 tests pour les services produit, panier et commande.	<i>Tests</i>

H — Concepts E-Commerce

Catalogue	Ensemble des produits disponibles a la vente, organises par categories. Dans E-Store, le catalogue est gere par le domaine 'catalog' (Product, Category).	<i>E-Commerce / Domaine</i>
Panier (Cart)	Liste temporaire des articles selectionnes par un utilisateur avant achat. Dans E-Store, le panier est gere en base de donnees et lie au compte utilisateur.	<i>E-Commerce / Domaine</i>

Commande (Order)	Achat finalise par un client. Elle contient la liste des articles, les quantites, le total et le statut (PENDING, CONFIRMED, SHIPPED, DELIVERED).	<i>E-Commerce / Domaine</i>
Inventaire (Inventory)	Gestion du stock disponible pour chaque produit. Lors d'une commande, le stock est decremente. Si le stock est insuffisant, la commande est refusee.	<i>E-Commerce / Domaine</i>
Avis (Review)	Commentaire et note laisses par un client sur un produit. Dans E-Store, les avis sont stockes dans MongoDB (document flexible).	<i>E-Commerce / Domaine</i>
Checkout	Processus de finalisation d'une commande : verification du stock, creation de la commande, decrementation de l'inventaire, vidage du panier. Execute dans une seule transaction ACID.	<i>E-Commerce / Process</i>
Categorie	Groupe de produits ayant des caracteristiques communes. Exemple : Electronique, Vetements, Livres. Permet de filtrer et naviguer dans le catalogue.	<i>E-Commerce / Catalogue</i>

Fin du Glossaire

Projet E-Store — Module Full-Stack — FSBM, Université Hassan II de Casablanca
Akram Belmoussa & Nouhaila Ben Soumane — Encadrant : Pr. Omar ZAHOUR — 2025/2026